

EA-0062 - AQELYN Engineering Archive

EA-0062 - AQELYN Engineering Portal & Mission Control

****Subtitle:**** Developer Portal, Mission Control, Codex Integration, Claude Code Integration, GitHub/CI/CD Integration

****Status:**** FULL_COMPLETE

****Date:**** 2026-07-09

****Baseline:**** AQELYN Platform Enterprise Architecture Baseline v1.0

1. Executive Summary

EA-0062 defines the AQELYN Engineering Portal & Mission Control, the single operational interface for developers, AI coding agents, architects, testers, and release managers working on the AQELYN Platform.

The portal consolidates architecture, Engineering Archives, implementation status, module dependency tracking, coding standards, UI/UX standards, AI engineering rules, CI/CD health, test coverage, quality gates, Architecture Decision Records, release plans, and AI agent instructions into one authoritative environment.

The purpose is simple: every human developer and every AI coding agent starts from the same place, follows the same standards, implements the same EA sequence, and updates the same implementation status. This prevents architectural drift and turns AQELYN from a document-heavy project into an executable engineering program.

EA-0062 does not replace the existing Engineering Archives. It provides a browser-based operating layer above them.

2. Scope

In scope:

- Developer Portal web application.
- Mission Control dashboard.
- Engineering Archive browser.
- START_HERE workflow integration.
- Implementation status dashboard.
- Dependency graph viewer.
- Requirements and traceability viewer.
- Codex task-generation workflow.
- Claude Code task-generation workflow.
- Cursor/GitHub Copilot guidance pages.
- GitHub repository integration.
- CI/CD integration.

- Test results and quality-gate dashboards.
- ADR library.
- Release readiness dashboard.
- Design System browser.
- AI prompt library browser.
- Documentation search.
- Role-based access control.

Out of scope:

- Replacing GitHub or source control.
- Replacing CI/CD runners.
- Replacing the product runtime.
- Rewriting existing Engineering Archives.
- Public customer-facing product UI.

3. Engineering Objectives

OBJ-0062-001: Provide a single entry point for all AQELYN engineering work.

OBJ-0062-002: Make the implementation order explicit and enforceable.

OBJ-0062-003: Provide AI coding agents with precise, bounded task instructions.

OBJ-0062-004: Integrate Codex and Claude Code workflows without requiring architecture redesign.

OBJ-0062-005: Expose module status, test status, quality gates, and release readiness.

OBJ-0062-006: Maintain traceability from EA requirements to code, tests, and review artifacts.

OBJ-0062-007: Support architecture governance and change-control workflows.

OBJ-0062-008: Make all engineering documentation searchable and navigable.

OBJ-0062-009: Provide clear separation between architecture, implementation, testing, and release states.

OBJ-0062-010: Prevent AI agents from inventing architecture outside the approved EA baseline.

4. Product Principle Alignment

The portal implements the AQELYN Product Principles directly:

Explain Before You Recommend: every engineering action page explains what the developer must do, why it matters, what evidence supports it, and what acceptance criteria apply.

Simplicity First: the portal presents architecture and implementation status visually, not as a scattered folder of files.

Evidence Before Opinion: every implementation decision links to Engineering Archives, requirements, traceability entries, tests, and review artifacts.

Human-Centered Security: the portal reduces uncertainty for developers and AI agents.

Expert Depth on Demand: summary dashboards link to full EA packages, API contracts, database models, prompts, test logs, and ADRs.

5. High-Level Architecture

The Engineering Portal is a modular web application built as part of AQELYN's internal engineering toolchain.

Recommended runtime:

- Frontend: React + TypeScript.
- Backend: FastAPI + Python.
- Database: PostgreSQL for structured state.
- Search: OpenSearch or PostgreSQL full text search for documentation search.
- Graph: optional Neo4j or graph projection from PostgreSQL for dependency view.
- Authentication: OIDC-compatible authentication.
- Authorization: RBAC with engineering roles.
- CI/CD integration: GitHub Actions first, extensible later.
- AI integrations: Codex task prompts, Claude Code task prompts, Cursor rules, GitHub Copilot instructions.

The portal does not execute production security scanning. It governs engineering activity and implementation readiness.

6. System Components

Primary components:

1. Portal Shell - navigation, layout, authentication state, and global search.
2. Architecture Browser - Project Charter, standards, EA packages, indexes, and diagrams.
3. EA Implementation Tracker - EA-by-EA status, owners, tests, review state, and dependencies.
4. Mission Control Dashboard - overall implementation health.
5. Dependency Graph Viewer - module order and dependency visualization.
6. AI Agent Workbench - Codex, Claude Code, Cursor, and Copilot instruction generator.
7. Requirements Explorer - requirement status and acceptance criteria.
8. Traceability Explorer - requirement-to-code-to-test-to-review mapping.
9. CI/CD Health Dashboard - builds, tests, coverage, lint, security scans, and releases.
10. ADR Library - architecture decision records.
11. Design System Browser - UI standards from EA-0059.
12. AI Prompt Library Browser - AI standards from EA-0060.
13. Release Dashboard - readiness gates and version status.
14. Admin Console - roles, permissions, integrations, and portal settings.

7. Mission Control Dashboard

Mission Control is the primary landing page.

It shall show:

- Architecture status.
- Implementation progress.
- Testing progress.
- Security status.
- Documentation completeness.
- Current EA in implementation.
- Current sprint.
- Open blockers.
- Build status.
- Quality gate status.
- Release readiness.

Example summary:

Architecture: 100%

Implementation: 0%

Testing: 0%

Documentation: 100%

Current task: EA-0001 AQELYN Kernel

Build: Not started

Release: Not ready

Mission Control must be simple enough for project owners and detailed enough for engineers.

8. Engineering Archive Browser

The EA Browser provides structured access to every Engineering Archive.

Each EA page shall include:

- EA number.
- Title.
- Implementation status.
- Dependencies.
- Requirements Matrix.
- Traceability Matrix.
- Architecture diagrams.
- API contracts.
- Testing requirements.
- Acceptance criteria.
- Download links to PDF, HTML, Markdown, and ZIP.
- AI-agent task generation.

The browser must not allow undocumented architecture changes. If a change is required, it must produce a change request that becomes a future EA package.

9. Implementation Status Model

Each EA implementation shall have a controlled state.

States:

- Not Started
- Reading
- Planned
- In Progress
- Code Complete
- Unit Tested
- Integration Tested
- Security Reviewed
- Documentation Updated
- Traceability Updated
- Engineering Review
- Complete
- Blocked

Transitions must be auditable. The portal should record who or which agent changed the state and why.

10. Dependency Graph

The Dependency Graph visualizes the implementation order.

Core path:

EA-0001 AQELYN Kernel

-> EA-0002 Universal Object Model

-> EA-0003 Event Bus

-> EA-0004 Evidence Engine

-> EA-0005 Knowledge Graph

-> EA-0006 Trust Engine

-> EA-0007 Mission Engine

-> EA-0008 Workflow Engine

-> EA-0009 Policy Engine

-> Identity, endpoint, web, attack-surface, vulnerability, asset, AI, reporting, and automation engines.

The graph shall prevent starting a dependent module before required upstream modules are complete unless an approved exception exists.

11. Codex Integration

The Codex integration provides task packages for coding agents.

A Codex task package shall contain:

- Current EA.
- Scope boundary.
- Allowed files and directories.
- Required documents to read.
- Interfaces to implement.
- Tests to create.
- Definition of Done.
- Prohibited actions.
- Output expectations.

Codex must be instructed:

- Do not redesign architecture.
- Do not change repository structure.
- Implement only the current EA.
- Run tests.
- Update traceability.
- Produce implementation notes.

The portal should generate a copyable task prompt and a machine-readable JSON task file.

12. Claude Code Integration

Claude Code integration shall provide a parallel task workflow optimized for repository-aware implementation.

The portal shall generate Claude Code task bundles containing:

- START_HERE excerpt.
- Current EA summary.
- Relevant source files.
- Target implementation paths.
- Test paths.
- Review checklist.
- Guardrails.
- Expected commit boundaries.

Claude Code system guidance:

1. Read START_HERE.md.
2. Read EA-0058, EA-0059, EA-0060, and EA-0061.
3. Read the current EA package.
4. Inspect existing code before editing.
5. Implement the smallest compliant increment.

6. Run tests and static checks.
7. Update traceability and implementation status.
8. Never restructure the repository.
9. Never invent undocumented features.

Claude Code output shall include:

- Files changed.
- Tests run.
- Requirements satisfied.
- Known limitations.
- Next recommended EA task.

13. Cursor and GitHub Copilot Integration

The portal shall publish reusable AI coding rules for Cursor and GitHub Copilot.

Generated artifacts:

- .cursorrules or equivalent project instruction file.
- copilot-instructions.md.
- agent task templates.
- forbidden-actions checklist.
- EA implementation checklist.

These integrations must reinforce the same governance model as Codex and Claude Code.

14. GitHub Integration

The portal shall integrate with GitHub for repository state and development workflow.

Supported capabilities:

- Repository health summary.
- Branch status.
- Pull request status.
- Issues linked to EA requirements.
- Commits linked to implementation tasks.
- Release tags.
- GitHub Actions workflow results.
- Security alerts where available.

Every PR should reference the EA and requirement IDs it implements.

15. CI/CD Integration

The CI/CD dashboard shall show:

- Build status.
- Unit test results.
- Integration test results.
- E2E test results.
- Code coverage.
- Lint results.
- Type-check status.
- Dependency scan status.
- Secret scan status.
- SAST status.
- Container scan status.
- Artifact generation status.

A module cannot be marked Complete unless relevant quality gates pass or an approved exception is recorded.

16. Role-Based Access Control

Required roles:

- Owner
- Architect
- Lead Developer
- Developer
- AI Agent
- QA Engineer
- Security Reviewer
- Release Manager
- Auditor
- Viewer

Key rules:

- AI Agent may update implementation notes but must not approve architecture changes.
- Developer may mark code complete but not security review complete.
- Security Reviewer controls security review gates.
- Architect controls architecture change requests.
- Owner controls baseline freeze and release approval.

17. Data Model

Core entities:

- EngineeringArchive

- Requirement
- TraceabilityLink
- ImplementationTask
- Module
- Dependency
- TestRun
- QualityGate
- BuildRun
- PullRequest
- ArchitectureDecisionRecord
- ChangeRequest
- AgentInstructionBundle
- ReleaseCandidate
- User
- Role
- Permission

Every entity shall include `created_at`, `updated_at`, `source_reference`, and audit metadata.

18. API Specification

Representative backend APIs:

GET `/api/v1/archives`

GET `/api/v1/archives/{ea_id}`

GET `/api/v1/archives/{ea_id}/requirements`

GET `/api/v1/archives/{ea_id}/traceability`

GET `/api/v1/implementation/status`

PATCH `/api/v1/implementation/{ea_id}/status`

GET `/api/v1/dependencies`

GET `/api/v1/mission-control`

POST `/api/v1/agents/codex/task`

POST `/api/v1/agents/claude-code/task`

POST `/api/v1/agents/cursor/rules`

GET `/api/v1/ci/status`

GET `/api/v1/releases/readiness`

POST `/api/v1/change-requests`

All APIs shall follow EA-0058 naming, error handling, logging, authentication, and observability rules.

19. UI Specification

Primary pages:

- /start
- /mission-control
- /architecture
- /archives
- /archives/:eald
- /implementation
- /dependencies
- /requirements
- /traceability
- /ai-agents
- /ai-agents/codex
- /ai-agents/claude-code
- /ci-cd
- /quality
- /adr
- /design-system
- /ai-handbook
- /release
- /admin

UI must follow EA-0059. Dashboard cards should use clear statuses, minimal visual noise, accessible color contrast, and progressive disclosure.

20. Security Architecture

Security controls:

- OIDC authentication.
- RBAC authorization.
- Audit logging for state changes.
- Signed release status records.
- Read-only architecture baseline unless user has Architect role.
- AI task generation sandboxing.
- No secrets stored in generated prompts.
- GitHub tokens stored only in approved secret storage.
- Sensitive integration tokens never exposed in frontend state.
- Rate limiting on mutation endpoints.

The portal is an internal engineering system and must be treated as a high-value governance application.

21. Observability

Required telemetry:

- API request logs.
- State transition logs.
- AI task generation logs.
- Integration sync logs.
- Failed authentication attempts.
- CI/CD synchronization health.
- Search index freshness.
- Document parsing failures.
- Dashboard load times.
- User activity audit trails.

All logs shall include correlation_id, actor_id, component, operation, outcome, and timestamp.

22. Testing Strategy

Testing levels:

- Unit tests for status transitions, dependency resolution, RBAC, and task generation.
- Integration tests for GitHub, CI/CD, document indexing, and API flows.
- UI tests for navigation, dashboard rendering, filters, and accessibility.
- Security tests for authorization boundaries and token handling.
- AI prompt tests validating that generated instructions preserve architecture constraints.

Acceptance requires deterministic generation of Codex and Claude Code task bundles from the same EA input.

23. Acceptance Criteria

EA-0062 is accepted when:

- Portal shell renders successfully.
- START_HERE workflow is available.
- EA Browser lists EA-0001 through EA-0062.
- Mission Control shows architecture and implementation state.
- Dependency Graph displays core implementation order.
- Codex task bundle generation works.
- Claude Code task bundle generation works.
- GitHub integration contract is defined.
- CI/CD integration contract is defined.
- Requirements and traceability views work from structured data.
- RBAC is enforced.
- Audit logging exists for state transitions.
- Tests pass.

- Documentation is complete.

24. Requirements Matrix

REQ-0062-001: The system shall provide a single developer portal landing page.

REQ-0062-002: The system shall display EA implementation status.

REQ-0062-003: The system shall enforce EA dependency ordering.

REQ-0062-004: The system shall generate Codex task bundles.

REQ-0062-005: The system shall generate Claude Code task bundles.

REQ-0062-006: The system shall expose GitHub integration status.

REQ-0062-007: The system shall expose CI/CD quality gates.

REQ-0062-008: The system shall provide requirements and traceability navigation.

REQ-0062-009: The system shall enforce RBAC.

REQ-0062-010: The system shall audit state changes.

REQ-0062-011: The system shall provide architecture change request workflow.

REQ-0062-012: The system shall integrate EA-0058 through EA-0061 standards.

25. Traceability Matrix

REQ-0062-001 -> Portal Shell -> UI test -> Portal render evidence.

REQ-0062-002 -> Implementation Tracker -> API/unit tests -> Status data evidence.

REQ-0062-003 -> Dependency Graph -> Graph tests -> Blocked dependency evidence.

REQ-0062-004 -> AI Agent Workbench -> Prompt generation tests -> Codex task bundle.

REQ-0062-005 -> AI Agent Workbench -> Prompt generation tests -> Claude Code task bundle.

REQ-0062-006 -> GitHub Integration -> Integration tests -> Repository status evidence.

REQ-0062-007 -> CI/CD Dashboard -> Integration tests -> Build status evidence.

REQ-0062-008 -> Requirements Explorer -> UI/API tests -> Requirement navigation evidence.

REQ-0062-009 -> Auth/RBAC Layer -> Security tests -> Authorization evidence.

REQ-0062-010 -> Audit Service -> Unit/integration tests -> Audit log evidence.

REQ-0062-011 -> Governance Workflow -> Functional tests -> Change request evidence.

REQ-0062-012 -> Standards Integration -> Review checklist -> Standards compliance evidence.

26. Engineering Journal

Decision: Create EA-0062 as an internal engineering system rather than a static documentation page.

Rationale: AQELYN has a large architecture baseline. A portal reduces navigation burden, improves AI-agent compliance, and keeps implementation status traceable.

Decision: Include both Codex and Claude Code integration.

Rationale: The project will likely use multiple AI coding agents. They require consistent constraints and task boundaries.

Decision: Portal state must be auditable.

Rationale: Engineering status, quality gates, and review state affect implementation trust.

Open consideration: first implementation can be static-document backed, with database integration added incrementally.

27. Implementation Roadmap

Phase 1: Static Portal Prototype

- React shell.
- Local JSON data for EA indexes.
- Mission Control view.
- EA Browser.
- START_HERE page.

Phase 2: Backend API

- FastAPI services.
- PostgreSQL schema.
- Status update API.
- RBAC foundation.

Phase 3: AI Agent Workbench

- Codex prompt bundles.
- Claude Code prompt bundles.
- Cursor and Copilot instruction exports.

Phase 4: GitHub/CI Integration

- Repository status.
- PR status.
- GitHub Actions results.

Phase 5: Governance and Release

- Change requests.
- ADR workflow.
- Release readiness dashboard.

28. Final Review

EA-0062 completes the architecture for the AQELYN Engineering Portal & Mission Control.

It is a governance and productivity layer, not a customer-facing feature. It should be implemented after the first foundation modules exist or as a parallel internal tool once EA-0001 through EA-0003 are underway.

The portal shall become the operational entry point for human developers and AI coding agents throughout AQELYN implementation.

